

Data Science Libraries Interview Prep Guide

NumPy · Pandas · Seaborn · Matplotlib

High-probability interview questions with human-friendly answers and code

38+
Total Questions

19
High Priority

4
Libraries Covered

Code
Every Answer

Contents

NumPy

11 questions | 6 high priority | 5 medium priority

1. What is a NumPy ndarray and how does it differ from a Python list? [HIGH]
2. Explain broadcasting in NumPy with a clear example. [HIGH]
3. Difference between a view and a copy in NumPy. Why does it matter? [HIGH]
4. What does axis=0 vs axis=1 mean? How to remember it? [HIGH]
5. Explain np.where() and give a practical use case. [HIGH]
6. reshape vs ravel vs flatten vs resize — clear differences? [MED]
7. What is vectorization and why is it 10-100x faster than loops? [HIGH]
8. np.dot vs np.matmul vs @ operator — when to use which? [MED]
9. How do you stack arrays? np.vstack, hstack, concatenate? [MED]
10. What are universal functions (ufuncs)? Give examples. [MED]
11. How to generate random arrays? What is np.random.seed for? [MED]

Pandas

11 questions | 5 high priority | 6 medium priority

1. loc vs iloc — what is the exact difference? [HIGH]
2. How do you handle missing values in Pandas? [HIGH]
3. Explain groupby().agg() with a real example. [HIGH]
4. merge vs join vs concat — difference and when to use each? [HIGH]
5. apply() vs map() vs applymap() — when to use which? [HIGH]
6. What is a MultiIndex? How do you work with it? [MED]
7. How does pivot_table work? Difference from groupby? [MED]
8. How do you optimize Pandas for large datasets? [MED]
9. Difference between Series and DataFrame? [MED]
10. How do you handle duplicates in a DataFrame? [MED]
11. How do you work with datetime data in Pandas? [MED]

Seaborn

8 questions | 5 high priority | 3 medium priority

1. Seaborn vs Matplotlib — when to use which? [HIGH]
2. What do hue, style, and size parameters do in Seaborn? [HIGH]
3. When would you use pairplot? What does it show? [HIGH]
4. How do you plot a correlation heatmap? Best practices? [HIGH]
5. distplot vs histplot vs kdeplot — differences? [HIGH]
6. What is the difference between axes-level and figure-level functions in Seaborn? [MED]
7. How do you customize Seaborn themes and styles? [MED]
8. How do you plot a regression line with scatter in Seaborn? [MED]

Matplotlib

8 questions | 3 high priority | 5 medium priority

1. What is the difference between Figure and Axes in Matplotlib? [HIGH]
2. How do you create subplots with different sizes using GridSpec? [HIGH]
3. How do you save a figure at high resolution? [HIGH]
4. plt.plot() vs plt.scatter() — which to use when? [MED]
5. How do you add a twin y-axis for dual-scale charts? [MED]
6. How do you add annotations to a Matplotlib plot? [MED]
7. How do you customize tick labels, rotation, and limits? [MED]
8. How do you plot a bar chart and customize it properly? [MED]

NumPy

Q01

HIGH PRIORITY

What is a NumPy ndarray and how does it differ from a Python list?

ANSWER

ndarray (n-dimensional array) is NumPy's core data structure. It stores elements of the same dtype in a contiguous block of memory, which makes it blazing fast for numerical operations. Python lists are heterogeneous, dynamic, and slower because each element is a Python object with overhead. For ML/data work, always prefer ndarray.

```
import numpy as np
a = np.array([1, 2, 3]) # dtype auto-detected: int64
print(a.dtype, a.shape) # int64 (3,)
print(a.itemsize) # 8 bytes per element
```

Resume tip: Resume hook: image arrays in PlantCareAI were ndarray — each pixel = float value.

Q02

HIGH PRIORITY

Explain broadcasting in NumPy with a clear example.

ANSWER

Broadcasting is NumPy's ability to perform arithmetic on arrays of different shapes without physically copying data. The smaller array is 'stretched' along size-1 dimensions to match the larger. Rule: shapes are compared right-to-left; dimensions must be equal OR one of them must be 1.

```
a = np.array([[1,2,3],[4,5,6]]) # shape (2, 3)
b = np.array([10, 20, 30]) # shape (3,) broadcasts to (2,3)
print(a + b)
# [[11 22 33]
#  [14 25 36]]
```

Q03

HIGH PRIORITY

Difference between a view and a copy in NumPy. Why does it matter?

ANSWER

A view shares the same underlying data buffer — modifying it changes the original array. A copy is completely independent. Slices with basic indexing (`a[1:3]`) return views. Fancy indexing (`a[[1,2]]`) returns copies. This matters for memory efficiency and avoiding accidental data mutations in ML pipelines.

```
a = np.array([1,2,3,4,5])
v = a[1:4] # view – shares memory
v[0] = 99
print(a) # [1, 99, 3, 4, 5] – original changed!
c = a[1:4].copy() # independent copy
c[0] = 0
print(a) # unchanged
```

Q04

HIGH PRIORITY

What does `axis=0` vs `axis=1` mean? How to remember it?

ANSWER

`axis=0` runs down the rows (vertical direction). Operations along `axis=0` collapse rows and give a column-wise result. `axis=1` runs across the columns (horizontal direction). Operations along `axis=1` collapse columns and give a row-wise result. Memory trick: `axis=0` means 'going down', `axis=1` means 'going across'.

```
a = np.array([[1,2],[3,4]])
np.sum(a, axis=0) # [4, 6] – sum down each column
np.sum(a, axis=1) # [3, 7] – sum across each row
np.mean(a, axis=0) # [2., 3.] – mean of each column
```

Q05

HIGH PRIORITY

Explain `np.where()` and give a practical use case.

ANSWER

`np.where(condition, x, y)` works like a vectorized if-else. Where condition is True it returns x, else y. Used heavily for conditional transformations, label assignments, and masking — all without any Python loop.

```
scores = np.array([45, 72, 30, 88, 55])
result = np.where(scores >= 50, 'pass', 'fail')
# ['fail', 'pass', 'fail', 'pass', 'pass']
# Also: get indices where condition is True
idx = np.where(scores > 60)
print(scores[idx]) # [72 88]
```

Q06

IMPORTANT

reshape vs ravel vs flatten vs resize — clear differences?

ANSWER

reshape: returns new shape, same data (view if possible, must match total size). ravel: returns 1D, view if possible. flatten: always returns 1D copy. resize: in-place shape change — can repeat data or truncate if size changes.

```
a = np.array([[1,2,3],[4,5,6]])
a.reshape(6) # [1 2 3 4 5 6] – view
a.ravel() # [1 2 3 4 5 6] – view
a.flatten() # [1 2 3 4 5 6] – copy
a.resize(3, 2) # [[1 2][3 4][5 6]]
```

Q07

HIGH PRIORITY

What is vectorization and why is it 10-100x faster than loops?

ANSWER

Vectorization means applying operations to entire arrays at once instead of element-by-element Python loops. NumPy calls optimized C/Fortran code under the hood — no Python interpreter overhead per element. For 1 million elements, a vectorized op takes ~1ms vs ~500ms for a Python loop.

```
# Slow: Python loop
result = [x**2 for x in range(1_000_000)]
# Fast: vectorized
a = np.arange(1_000_000)
result = a ** 2 # ~100x faster
# Timing check
import time
# vectorized version finishes in ~1ms
```

Resume tip: Resume hook: all preprocessing in PlantCareAI (normalization, pixel ops) was vectorized.

Q08

IMPORTANT

np.dot vs np.matmul vs @ operator — when to use which?**ANSWER**

For 2D matrices all three do matrix multiplication. np.dot on 1D arrays returns scalar dot product. np.matmul and @ do NOT support scalar multiplication (raise error). For ML weight matrix operations, prefer @ (clean syntax).

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
A @ B # matrix multiply – preferred
np.matmul(A, B) # same result
np.dot(A, B) # same for 2D
# 1D dot product
np.dot([1,2,3], [4,5,6]) # 32 (scalar)
```

Q09

IMPORTANT

How do you stack arrays? np.vstack, hstack, concatenate?**ANSWER**

np.vstack: stack vertically (row-wise, axis=0). np.hstack: stack horizontally (column-wise, axis=1). np.concatenate: general-purpose, specify axis explicitly. np.stack: creates a NEW axis (unlike concat).

```
a = np.array([1,2,3])
b = np.array([4,5,6])
np.vstack([a, b]) # [[1 2 3],[4 5 6]] shape (2,3)
np.hstack([a, b]) # [1 2 3 4 5 6] shape (6,)
np.stack([a, b], axis=0) # [[1 2 3],[4 5 6]] new axis
```

Q10

IMPORTANT

What are universal functions (ufuncs)? Give examples.**ANSWER**

Ufuncs (universal functions) are NumPy functions that operate element-wise on arrays. They're implemented in C for speed and support broadcasting automatically. Examples: np.add, np.multiply, np.sqrt, np.exp, np.log, np.sin, np.maximum.

```
a = np.array([1, 4, 9, 16])
np.sqrt(a) # [1. 2. 3. 4.]
np.exp(a) # [2.718, 54.6, 8103, ...]
np.log(a) # [0, 1.386, 2.197, 2.773]
# maximum element-wise (not max)
np.maximum([1,5,3], [2,4,6]) # [2, 5, 6]
```

How to generate random arrays? What is np.random.seed for?

ANSWER

np.random provides functions to generate random data. seed() fixes the random state so results are reproducible — critical for ML experiments and debugging.

```
np.random.seed(42) # reproducible
np.random.rand(3, 3) # uniform [0, 1)
np.random.randn(3, 3) # standard normal
np.random.randint(0, 10, (3, 3)) # integers 0-9
np.random.choice([1,2,3,4], 5) # random sample
```

Resume tip: Always set seed before any ML experiment for reproducibility.

Pandas

Q01

HIGH PRIORITY

loc vs iloc — what is the exact difference?

ANSWER

loc uses label-based indexing (row/column names or boolean arrays). iloc uses integer position-based indexing (0, 1, 2...). Key gotcha: loc slicing is INCLUSIVE on both ends. iloc slicing excludes the end like Python lists. When the index is a RangeIndex (default), they look the same but behave differently for slices.

```
df.loc[0:3, 'Age'] # rows labeled 0,1,2,3 – inclusive!
df.iloc[0:3, 2] # rows at position 0,1,2 – end excluded
# Boolean indexing – only loc
df.loc[df['Age'] > 25, ['Name', 'Age']]
# Single value retrieval
df.at[0, 'Age'] # fastest for single label
df.iat[0, 2] # fastest for single position
```

Resume tip: Resume hook: used in EDA for Car Sales Dashboard — loc for condition-based filtering.

Q02

HIGH PRIORITY

How do you handle missing values in Pandas?

ANSWER

Step 1: Detect — isnull() or isna() (same thing). Step 2: Decide strategy — drop (dropna), fill with fixed value, fill with mean/median, or forward/backward fill. For numerical columns in ML, median imputation is safer than mean (less affected by outliers).

```
df.isnull().sum() # count NaN per column
df.isnull().sum() / len(df) * 100 # % missing
df.dropna(subset=['Age']) # drop rows where Age is NaN
df['Age'].fillna(df['Age'].median(), inplace=True)
df.fillna(method='ffill') # forward fill
df.fillna(method='bfill') # backward fill
```


Q03

HIGH PRIORITY

Explain `groupby().agg()` with a real example.

ANSWER

`groupby()` splits the DataFrame by a key column. `agg()` applies multiple aggregation functions in one shot, returning a clean summary DataFrame. You can pass a dict to apply different functions to different columns.

```
result = df.groupby('Region').agg(  
    Total_Sales = ('Sales', 'sum'),  
    Avg_Price = ('Price', 'mean'),  
    Max_Units = ('Units', 'max'),  
    Count = ('ID', 'count')  
) .reset_index()  
# Multiple keys  
df.groupby(['Region', 'Brand'])['Sales'].sum()
```

Resume tip: Resume hook: Car Sales Dashboard — regional brand-wise sales breakdown used `groupby.agg()`.

Q04

HIGH PRIORITY

merge vs join vs concat — difference and when to use each?

ANSWER

`merge`: SQL-style join on specific columns, very flexible (inner/left/right/outer). `join`: shorthand for index-based merge. `concat`: stacks DataFrames row-wise or column-wise without key matching. For combining datasets from different sources, `merge` is your go-to.

```
# merge on column (SQL style)  
pd.merge(orders, customers, on='CustomerID', how='left')  
# join on index  
df1.join(df2, how='inner')  
# concat — stack rows  
pd.concat([df_q1, df_q2, df_q3], axis=0, ignore_index=True)  
# concat — add columns  
pd.concat([df_features, df_labels], axis=1)
```

Q05

HIGH PRIORITY

apply() vs map() vs applymap() — when to use which?

ANSWER

map(): Series only — element-wise transformation or substitution using a dict/function. apply(): Series or DataFrame — applies a function along axis=0 (rows) or axis=1 (columns). applymap() / DataFrame.map() (Pandas 2.1+): element-wise on every cell of a DataFrame. For performance, always prefer built-in vectorized ops over apply() when possible.

```
# map – substitution on Series
df['Grade'] = df['Score'].map({90:'A', 80:'B', 70:'C'})
# apply – row/column wise
df['Name'] = df['Name'].apply(lambda x: x.strip().title())
df['BMI'] = df.apply(lambda r: r['Weight']/r['Height']**2, axis=1)
# applymap – every cell
df.applymap(lambda x: round(x, 2))
```

Q06

IMPORTANT

What is a MultiIndex? How do you work with it?

ANSWER

MultiIndex is a hierarchical index with 2+ levels. It commonly appears after groupby with multiple keys. You can access data using .loc with tuples. Flatten it with reset_index() or droplevel().

```
result = df.groupby(['Region', 'Brand'])['Sales'].sum()
# MultiIndex Series – two-level index
result.loc['North', 'Toyota'] # access specific combo
result.reset_index() # flatten to normal DataFrame
result.unstack() # pivot to wide format
```

Q07

IMPORTANT

How does pivot_table work? Difference from groupby?

ANSWER

pivot_table is like groupby but presents the result as a 2D cross-tabulation — rows are one category, columns are another, cell values are aggregated. groupby is more flexible for multiple outputs. pivot_table is ideal for cross-comparison reports.

```
pd.pivot_table(
    df,
    values = 'Sales',
    index = 'Region',
    columns = 'Brand',
    aggfunc = 'sum',
    fill_value = 0
)
```

Resume tip: Resume hook: Car Sales Dashboard cross-analysis (Region x Brand x Sales).

Q08

IMPORTANT

How do you optimize Pandas for large datasets?

ANSWER

Use category dtype for low-cardinality string columns (e.g., gender, region). Use int32/float32 instead of 64-bit when precision allows. Use chunksize in read_csv for files too large to fit in RAM. Avoid iterrows — use vectorized ops. Consider Polars or Dask for very large data.

```
# Reduce memory
df['Region'] = df['Region'].astype('category')
df['Age'] = df['Age'].astype('int32')
df['Price'] = df['Price'].astype('float32')
# Read large file in chunks
chunks = pd.read_csv('big_file.csv', chunksize=50_000)
result = pd.concat([process(c) for c in chunks])
```

Q09

IMPORTANT

Difference between Series and DataFrame?

ANSWER

Series is a 1D labeled array — a single column with an index. DataFrame is a 2D table — a collection of Series sharing the same index. A single column from a DataFrame (df['col']) returns a Series. A list of columns (df[['col']]) returns a DataFrame.

```
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
type(df['A']) # pandas.core.series.Series
type(df[['A']]) # pandas.core.frame.DataFrame
```

Q10

IMPORTANT

How do you handle duplicates in a DataFrame?

ANSWER

df.duplicated() returns a boolean Series marking duplicate rows. df.drop_duplicates() removes them. You can scope both to specific columns with subset. keep='first' (default) keeps first occurrence; keep='last' keeps last; keep=False marks all.

```
df.duplicated().sum() # count duplicates
df.duplicated(subset=['Name', 'Email']) # scope to columns
df.drop_duplicates(inplace=True) # remove duplicates
df.drop_duplicates(subset=['Email'], # keep first by Email
keep='first')
```

How do you work with datetime data in Pandas?

ANSWER

Convert to datetime with `pd.to_datetime()`. Then use `.dt` accessor to extract components. Setting datetime as index enables powerful time-series resampling and slicing.

```
df['Date'] = pd.to_datetime(df['Date'])
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['Weekday'] = df['Date'].dt.day_name()
# Resample: monthly average
df.set_index('Date').resample('M')['Sales'].mean()
```

Resume tip: Resume hook: time-series analysis in Car Sales Dashboard.

Seaborn

Q01

HIGH PRIORITY

Seaborn vs Matplotlib — when to use which?

ANSWER

Seaborn is built on top of Matplotlib. It provides high-level, stat-focused plots with beautiful default themes and tight Pandas integration. Use Seaborn for quick EDA (distributions, correlations, categorical comparisons). Use Matplotlib when you need fine-grained custom control — annotations, complex subplots, custom patches.

```
import seaborn as sns
import matplotlib.pyplot as plt
# Seaborn: quick categorical plot
sns.boxplot(data=df, x='Region', y='Sales', hue='Brand')
plt.title('Sales by Region')
plt.show()
```

Q02

HIGH PRIORITY

What do hue, style, and size parameters do in Seaborn?

ANSWER

These three parameters encode extra dimensions into a 2D plot. hue: maps a variable to point/line color. style: maps to marker shape or line dash style. size: maps to point or line size. Together they let you visualize 5 variables in one plot.

```
sns.scatterplot(
    data = df,
    x = 'Engine_CC',
    y = 'Price',
    hue = 'Brand',
    style = 'FuelType',
    size = 'Mileage'
)
plt.legend(bbox_to_anchor=(1,1))
```

Resume tip: Resume hook: used in Car Sales EDA to show multi-variable relationship.

Q03

HIGH PRIORITY

When would you use pairplot? What does it show?

ANSWER

pairplot shows pairwise scatter plots for all numeric column combinations, with distribution plots on the diagonal. It's a quick X-ray of your dataset — spot correlations, outliers, and class separability in one call. Used in the very first step of EDA.

```
sns.pairplot(
    df,
    hue = 'Target',
    diag_kind = 'kde',
    plot_kws = {'alpha': 0.6}
)
plt.suptitle('Pairwise Feature Relationships', y=1.02)
```

Resume tip: Resume hook: PlantCareAI EDA — check feature correlations before CNN training.

Q04

HIGH PRIORITY

How do you plot a correlation heatmap? Best practices?

ANSWER

Compute with `df.corr()`, then pass to `sns.heatmap()`. Use `annot=True` to show values, `fmt='.2f'` for 2 decimal places, `cmap='coolwarm'` for a diverging color scale (red=positive, blue=negative correlation), `vmin=-1`, `vmax=1` to normalize the scale.

```
corr_matrix = df.select_dtypes('number').corr()
plt.figure(figsize=(10, 8))
sns.heatmap(
    corr_matrix,
    annot = True,
    fmt = '.2f',
    cmap = 'coolwarm',
    vmin=-1, vmax=1,
    square = True,
    linewidths = 0.5
)
plt.title('Correlation Matrix')
```

Resume tip: Almost certain to be asked. Practice this from memory.

Q05

HIGH PRIORITY

distplot vs histplot vs kdeplot — differences?

ANSWER

distplot is deprecated since Seaborn 0.11. histplot is its modern replacement — shows histogram bars with optional KDE overlay. kdeplot shows only the smooth density estimate (no bars). histplot with kde=True is the go-to for showing a single variable's distribution.

```
# Modern approach
sns.histplot(df['Price'], kde=True, bins=30, color='steelblue')
# KDE only
sns.kdeplot(df['Price'], fill=True, alpha=0.5)
# Comparing two groups
sns.kdeplot(data=df, x='Price', hue='FuelType', fill=True)
```

Resume tip: Saying 'distplot is deprecated' shows you follow library updates.

Q06

IMPORTANT

What is the difference between axes-level and figure-level functions in Seaborn?

ANSWER

Axes-level functions (sns.boxplot, sns.scatterplot, sns.histplot) plot onto a single Axes — they work with plt.subplots() and give you full Matplotlib control. Figure-level functions (sns.catplot, sns.relplot, sns.displot) create their own Figure and support row/col faceting automatically. They return a FacetGrid object, not Axes.

```
# Axes-level – you control the figure
fig, axes = plt.subplots(1, 2)
sns.boxplot(data=df, x='Brand', y='Price', ax=axes[0])
# Figure-level – Seaborn manages the figure
g = sns.catplot(
    data=df, x='Brand', y='Price',
    col='Region', kind='box'
)
g.set_axis_labels('Brand', 'Price (INR)')
```

Q07

IMPORTANT

How do you customize Seaborn themes and styles?

ANSWER

`sns.set_theme()` or `sns.set_style()` controls the background grid style (darkgrid, whitegrid, dark, white, ticks). `sns.set_palette()` controls colors. `sns.set_context()` scales element sizes for different output targets (paper, notebook, talk, poster).

```
sns.set_theme(style='whitegrid', palette='muted', font_scale=1.2)
# or individually
sns.set_style('darkgrid')
sns.set_palette('husl')
sns.set_context('talk') # bigger fonts for presentations
# Reset
sns.reset_defaults()
```

Q08

IMPORTANT

How do you plot a regression line with scatter in Seaborn?

ANSWER

`sns.regplot()` adds a regression line with confidence interval to a scatter plot. `sns.lmplot()` is the figure-level version that supports hue and faceting. Both compute the regression internally — no scikit-learn needed.

```
# Simple regression plot
sns.regplot(data=df, x='Engine_CC', y='Price',
            scatter_kws={'alpha':0.5}, line_kws={'color':'red'})
# With hue (figure-level)
sns.lmplot(data=df, x='Engine_CC', y='Price',
           hue='FuelType', height=6)
```


Matplotlib

Q01

HIGH PRIORITY

What is the difference between Figure and Axes in Matplotlib?

ANSWER

Figure is the overall canvas — the entire window or image. Axes is the actual plot area inside the Figure — it contains the x-axis, y-axis, title, data, labels. One Figure can have multiple Axes (subplots). This is THE most fundamental concept — getting it wrong in an interview is a red flag.

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].plot([1,2,3], [4,5,6])
ax[0].set_title('Line Plot')
ax[0].set_xlabel('X')
ax[1].scatter([1,2,3], [6,5,4], color='red')
ax[1].set_title('Scatter')
plt.tight_layout()
plt.show()
```

Resume tip: If you say 'I just use plt.plot()' without explaining Figure/Axes, interviewer will probe harder.

Q02

HIGH PRIORITY

How do you create subplots with different sizes using GridSpec?

ANSWER

plt.subplots() with gridspec_kw handles simple ratio differences. GridSpec gives full control for complex layouts — spanning rows, spanning columns, custom gaps.

```
# Simple ratio approach
fig, axes = plt.subplots(1, 2, figsize=(12,4),
    gridspec_kw={'width_ratios': [2, 1]})
# GridSpec for complex layout
from matplotlib.gridspec import GridSpec
fig = plt.figure(figsize=(12, 6))
gs = GridSpec(2, 3, figure=fig)
ax1 = fig.add_subplot(gs[0, :]) # spans full top row
ax2 = fig.add_subplot(gs[1, 0:2]) # bottom left
ax3 = fig.add_subplot(gs[1, 2]) # bottom right
```

Resume tip: Resume hook: Car Sales Dashboard used GridSpec for layout — big trend chart + small KPIs.

Q03

HIGH PRIORITY

How do you save a figure at high resolution?

ANSWER

Use `fig.savefig()` or `plt.savefig()`. `dpi` controls resolution (300 for print/reports, 150 for web). `bbox_inches='tight'` is critical — without it, axis labels often get cut off. Supported formats: png, jpg, svg, pdf.

```
fig.savefig(
    'dashboard.png',
    dpi = 300,
    bbox_inches = 'tight',
    format = 'png'
)
# Transparent background (for presentations)
fig.savefig('chart.png', transparent=True, dpi=150)
```

Q04

IMPORTANT

plt.plot() vs plt.scatter() — which to use when?

ANSWER

`plot()` connects points with a line — ideal for time series and continuous data where order matters. `scatter()` plots independent points — ideal for correlation analysis, cluster visualization, and when you want per-point color/size control. `scatter()` is more memory-intensive for large datasets.

```
# Time series — use plot()
ax.plot(dates, monthly_sales, marker='o', linewidth=2)
# Correlation — use scatter()
ax.scatter(x, y,
    c = labels,
    cmap = 'viridis',
    alpha = 0.7,
    s = 60) # point size
```

Q05

IMPORTANT

How do you add a twin y-axis for dual-scale charts?

ANSWER

`ax.twinx()` creates a second Axes sharing the same x-axis with an independent y-axis. Essential for overlaying two metrics with very different scales — like sales volume and growth rate.

```
fig, ax1 = plt.subplots(figsize=(10, 5))
ax2 = ax1.twinx()
ax1.bar(months, sales, color='steelblue', alpha=0.7, label='Sales')
ax2.plot(months, growth_pct, color='red', marker='o', label='Growth %')
ax1.set_ylabel('Sales (INR)', color='steelblue')
ax2.set_ylabel('Growth %', color='red')
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1+lines2, labels1+labels2)
```

Resume tip: Resume hook: Car Sales Dashboard — sales volume + growth % on same chart.

Q06

IMPORTANT

How do you add annotations to a Matplotlib plot?

ANSWER

`ax.annotate()` adds text with an optional arrow pointing to a data point — great for highlighting max/min values. `ax.text()` adds plain text at coordinates. `ax.axhline()` / `ax.axvline()` add horizontal/vertical reference lines.

```
ax.annotate(
    'Peak Sales',
    xy = (peak_x, peak_y),
    xytext = (peak_x + 1, peak_y + 5000),
    arrowprops = dict(arrowstyle='->', color='red'),
    fontsize = 10
)
# Reference line
ax.axhline(y=df['Sales'].mean(), color='gray',
    linestyle='--', label='Average')
```

Q07

IMPORTANT

How do you customize tick labels, rotation, and limits?

ANSWER

`ax.set_xticks()` defines tick positions. `ax.set_xticklabels()` sets custom labels with rotation. `ax.set_xlim()` and `ax.set_ylim()` control axis range. `tick_params()` controls tick direction, size, and pad in one call.

```
ax.set_xticklabels(labels, rotation=45, ha='right', fontsize=9)
ax.set_ylim(0, df['Sales'].max() * 1.1) # 10% headroom
ax.tick_params(
    axis = 'x',
    direction = 'out',
    length = 5,
    labelsiz = 9
)
```

Q08

IMPORTANT

How do you plot a bar chart and customize it properly?

ANSWER

`ax.bar()` for vertical bars, `ax.barh()` for horizontal. Color by value using a colormap. Add value labels on top of bars. Grouped bars need manual x-offset with `np.arange()`.

```
brands = ['Toyota', 'Honda', 'Hyundai', 'Maruti']
sales = [5200, 4100, 3800, 6500]
bars = ax.bar(brands, sales,
    color=['#4ecdc4', '#f7b731', '#a29bfe', '#fd79a8'])
# Add value labels on bars
for bar in bars:
    h = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, h + 50,
        f'{h:,}', ha='center', va='bottom', fontsize=9)
```

Quick Revision Cheatsheet

NumPy

- ndarray = typed + contiguous memory = fast
- Broadcasting: shapes match right-to-left; dim must be equal or 1
- Slicing = view | Fancy indexing = copy
- axis=0 goes DOWN rows | axis=1 goes ACROSS columns
- np.where() = vectorized if-else — no loop needed
- Always set np.random.seed() before ML experiments

Pandas

- loc = label-based, inclusive ends | iloc = position-based, exclusive end
- groupby().agg() = split-apply-combine in one line
- merge > join > concat (decreasing flexibility)
- map() = Series substitution | apply() = row/column func | applymap() = cell-wise
- astype('category') can cut memory usage by up to 90% for string columns
- pd.to_datetime() + .dt accessor for all time-series work

Seaborn

- Seaborn = high-level wrapper over Matplotlib; tight Pandas integration
- hue=color | style=marker/dash | size=point size — encode 5D in 2D
- pairplot = quick full EDA in one line
- Correlation heatmap: df.corr() + sns.heatmap(annot=True, cmap='coolwarm')
- distplot is DEPRECATED — use histplot or kdeplot
- Axes-level = plug into subplots | Figure-level = built-in faceting

Matplotlib

- Figure = canvas | Axes = individual plot — one Figure, many Axes
- Always use fig.savefig(dpi=300, bbox_inches='tight') for production quality
- plot() = line/time-series | scatter() = correlation/distribution
- ax.twinx() for dual y-axis charts
- ax.annotate() to highlight key data points with arrows
- GridSpec for complex dashboard layouts with unequal subplots